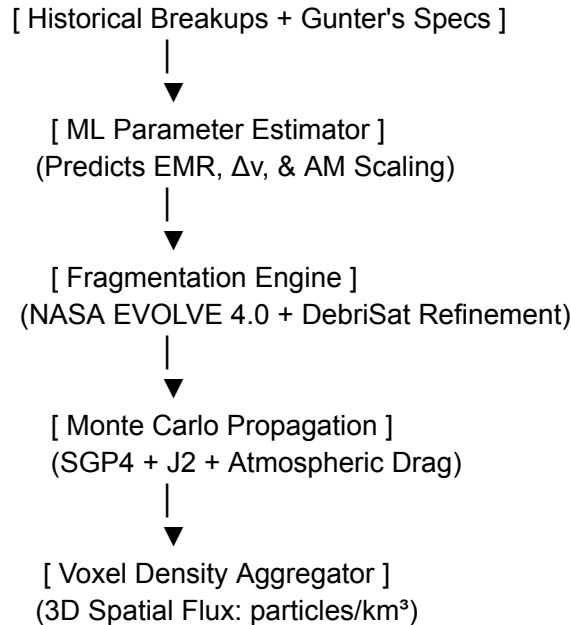


System Architecture

To build a realistic prediction model for untrackable debris clouds ($< 10cm$), avoid simulating every single microscopic particle individually over long horizons. Instead, structure your pipeline as a **Hybrid Physics-ML Cloud Density Generator**.



Phase 1: Feature Extraction Pipeline

You need to map satellite specs to physical breakup parameters. Build a structured dataset from historical breakup events.

Data Inputs

- **Gunter's Space Page Metadata:** Satellite dry mass (M_{dry}), fuel mass (M_{prop}), solar panel surface area (A_{panel}), bus structural volume (V_{bus}), primary structural material (aluminum alloy vs. carbon fiber composites).
- **Breakup Event Parameters:** Pre-impact orbits, relative collision velocity (v_{rel}), impact energy-to-mass ratio ($EMR = \frac{E_{kinetic}}{M_{target}}$ in J/g).
- **Historical Target Ground Truth:** Observed fragment counts for trackable objects from Space-Track/DISCOS, used to ground the total energy scale of the event.

Phase 2: The Fragmentation Engine (Physics + ML Hybrid)

Standard models like NASA's EVOLVE 4.0 assume older satellite construction (monolithic aluminum). Modern satellites (like those tested in the **DebrisSat** project) use multi-layer insulation (MLI), carbon-fiber panels, complex cabling, and dense circuit boards. DebrisSat showed that modern breakups produce **significantly higher total fragment counts** and a much higher proportion of high Area-to-Mass (A/M) fragments than the traditional NASA model predicted.

1. The Core Physics Equations

For a given characteristic length L_c (where $L_c < 0.1m$ for untrackable debris), calculate the cumulative number of fragments $N(L_c)$:

$$\text{Explosion: } N(L_c) = 6 \cdot (M_{gen})^{0.75} \cdot L_c^{-1.71}$$

$$\text{CatastrophicCollision}(EMR \geq 40J/g): N(L_c) = 0.1 \cdot (M_{total})^{0.75} \cdot L_c^{-1.71}$$

where M_{gen} is the ejected mass and M_{total} is the combined mass of impactor and target in kilograms.

2. DebrisSat-Informed A/M and Velocity (Δv) Distributions

Do not assume static fragment densities. Sample A/M (m^2/kg) and ejection velocities Δv (m/s) using distributions parameterized by L_c :

$$\log_{10}(\Delta v) = \alpha \cdot \log_{10}(A/M) + \beta + N(0, \sigma^2)$$

- **The ML Role:** Train an XGBoost or Random Forest model on your historical Gunter's dataset to predict the scalar shift factors (α, β) and total kinetic conversion efficiency based on the satellite's structural composition (e.g., carbon-fiber structures yield higher A/M fragments that decay much faster due to atmospheric drag).
-

Phase 3: Particle Cloud Propagation Engine

Instead of tracking millions of particles continuously, use a **Monte Carlo Sampling Strategy**:

1. **Sample K Synthetic Representative Particles:** Generate $K \approx 10,000$ synthetic particles distributed across logarithmic size bins from $2mm$ to $10cm$. Each synthetic particle represents a weight factor w_i of real fragments.
- 2.
3. **Ejection Vector Assignment:** Apply isotropic or directional velocity vectors $\vec{\Delta v}$ to the parent satellite's orbital velocity vector $\vec{V}_0$ at the exact moment of breakup:

4.

5.

$$6. \vec{V}_{particle} = \vec{V}_0 + \Delta v$$

7. **Vectorized Propagation Pipeline:**

8.

- Convert updated state vectors ($\vec{R}, \vec{V}$) to Two-Line Elements or Osculating Keplerian Elements.
-
- Propagate forward in time using SGP4 with J_2 perturbation (Earth oblateness) and atmospheric drag.
-
- **Drag Correction:** Calculate the B^* drag term for each particle based on its sampled A/M :
-
-

$$\circ B^* = \frac{C_D \cdot A}{2 \cdot m} \cdot \rho_0$$

- High A/M particles ($> 1m^2/kg$) will rapidly de-orbit or shift into lower orbital shells, causing the debris cloud to stretch along the velocity vector (forming a torus around Earth over 24–48 hours).
-

Phase 4: Spatial Cloud Density Generator (3D Voxel Grid)

To evaluate the cloud without individual particle rendering bottlenecks, map the cloud into a **4D Spatio-Temporal Volumetric Density Matrix** $D(x, y, z, t)$.

1. **Spatial Binning:** Divide the orbital shells (e.g., altitudes $300km - 1200km$) into a 3D Earth-Centered Inertial (ECI) grid or **HEALPix** / geodetic altitude-latitude-longitude voxels (e.g., $20km \times 20km \times 20km$).
- 2.
3. **Density Accumulation:** At any time step t , sum the particle weights w_i present inside voxel V_k :
- 4.
- 5.

$$6. \rho(V_k, t) = \frac{\sum_{i \in V_k} w_i}{Volume(V_k)} [particles/km^3]$$

7. **Gaussian Smoothing (Kernel Density Estimation):** Apply a 3D Gaussian blur kernel across neighboring voxels to account for positional uncertainty and smooth out Monte Carlo sampling artifacts.
- 8.

Phase 5: "Vibecoding" Stack & Execution Roadmap

To build this rapidly during a hackathon, use this modular Python setup:

```
cloud_predictor/
├── data/
│   ├── gunter_satellites.json    # Pre-scraped satellite specs
│   └── historical_breakups.csv    # Space-Track breakup events
├── engine/
│   ├── fragmentation.py          # NASA + DebrisSat vector equations
│   ├── propagator.py             # Vectorized SGP4 propagation
│   └── density_grid.py           # 3D Voxelization + KDE smoothing
├── ml/
│   └── train_breakup_scaler.py    # Predicts fragment multiplier
└── app.py                        # Fast API / Streamlit endpoint
```

Recommended Libraries

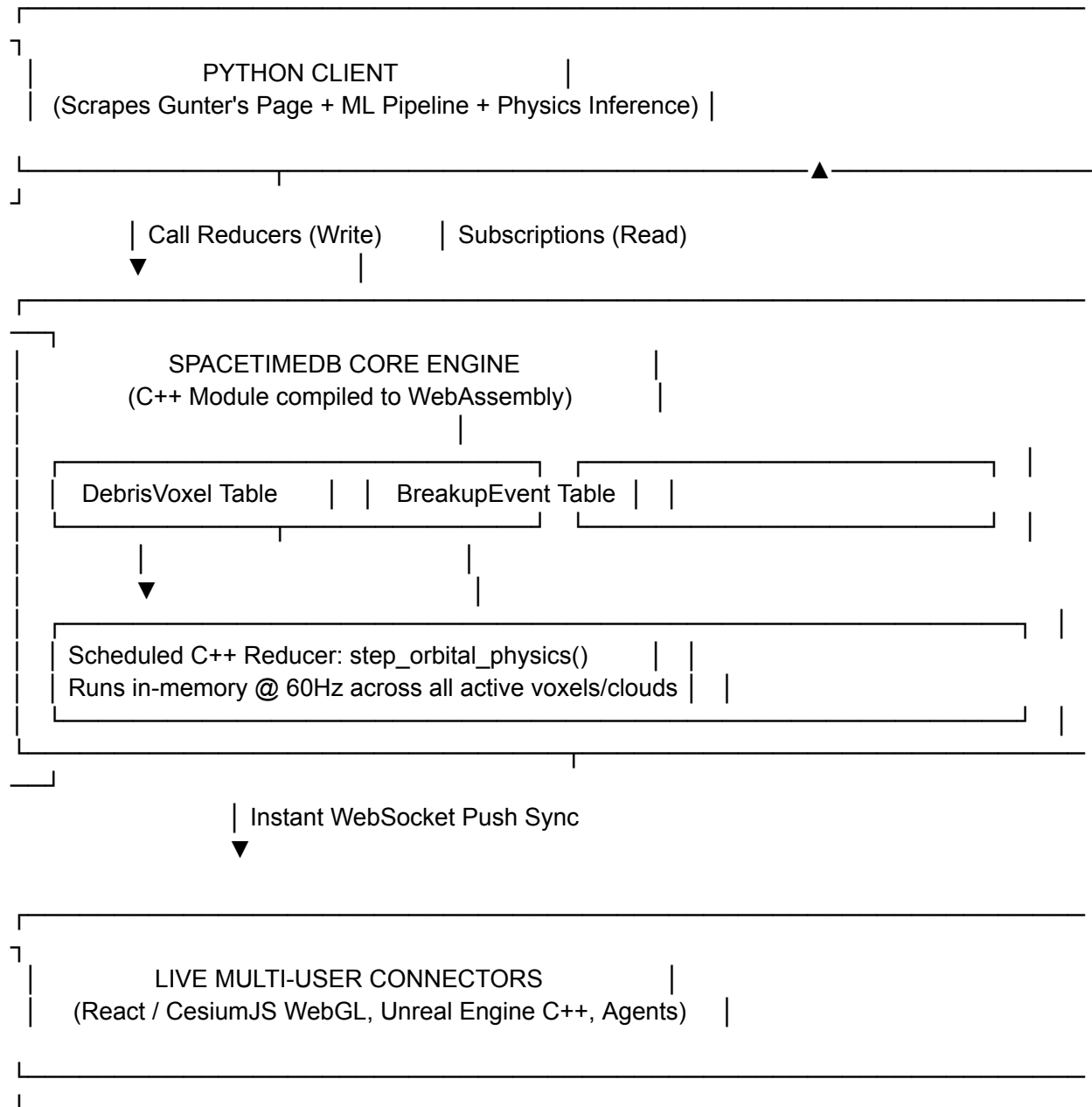
- **Propagation & Math:** `sgp4` (high-speed orbital calculations), `numpy` / `polars` (vectorized array operations), `scipy.spatial` (cKDTree for instant spatial indexing).
-
- **ML Model:** `xgboost` or `lightgbm` (fast training on tabular historical dataset).
-
- **Visualization (Front-End):** `Deck.gl` (SimpleMeshLayer / H3HexagonLayer) or `CesiumJS` to render 3D density heatmaps surrounding the globe.
-

Demo Strategy

In your presentation, showcase a timeline slider for a simulated catastrophic breakup (e.g., a collision in LEO):

1. $t = 0hrs$: Point-source explosion showing initial fragment ejection.
- 2.
3. $t = 6hrs$: Formation of a dense "debris band" along the parent trajectory.
- 4.
5. $t = 48hrs$: Dispersion into a full orbital shell torus, highlighting high-density "danger zones" that intersect common satellite operational corridors.

Using **SpacetimeDB** with C++ on the server (or native client) alongside Python for ML data pipeline tasks creates a hybrid architecture. SpacetimeDB acts as both the relational database and the real-time event broker.



Step 1: Write the Core C++ Backend Module

Initialize a C++ module using the SpacetimeDB CLI:

Bash

```
spacetime init --lang cpp spacetimedb
```

Inside spacetimedb/src/lib.cpp, define the shared state tables and the C++ transaction logic (reducers).

C++

```
#include "spacetimedb.h"
```

```
// 1. Define the shared state schema for spatial voxels
```

```
struct DebrisVoxel {  
    uint64_t voxel_id;  
    uint64_t event_id;  
    float x;  
    float y;  
    float z;  
    float vx;  
    float vy;  
    float vz;  
    float density;  
};
```

```
// Make the table public so connected Python scripts, web frontends, and agents can read it  
SPACETIMEDB_TABLE(DebrisVoxel, debris_voxel, Public);
```

```
// 2. Reducer: Invoked by Python to instantiate a calculated debris cloud
```

```
SPACETIMEDB_REducer(trigger_breakup, uint64_t event_id, float x, float y, float z, float  
mass_kg, float EMR) {
```

```
    // Generate initial spatial voxels within the database transaction
```

```
    for (uint64_t i = 0; i < 500; ++i) {  
        DebrisVoxel voxel;  
        voxel.voxel_id = (event_id * 10000) + i;  
        voxel.event_id = event_id;  
        voxel.x = x + (static_cast<float>(i) * 0.5f);  
        voxel.y = y;  
        voxel.z = z;  
        voxel.vx = 0.0f;  
        voxel.vy = 7.5f; // Initial orbital velocity vector (km/s)  
        voxel.vz = 0.1f;  
        voxel.density = (mass_kg * EMR) / (1.0f + static_cast<float>(i));
```

```
        // Insert directly into in-memory table
```

```
        debris_voxel::insert(voxel);
```

```
    }  
}
```

```
// 3. High-Frequency Physics Reducer: Advances orbital positions in real-time
```

```
SPACETIMEDB_REducer(step_orbital_physics, float dt) {
```

```
// Iterate over every active voxel in the database
for (auto voxel : debris_voxel::iter()) {
    // Velocity step (apply J2 perturbation / drag step here in C++)
    voxel.x += voxel.vx * dt;
    voxel.y += voxel.vy * dt;
    voxel.z += voxel.vz * dt;

    // Updating the table in C++ automatically streams the new coordinates
    // to every connected Python process, Web browser, and C++ client!
    debris_voxel::update_by_voxel_id(voxel.voxel_id, voxel);
}
}
```

Compile and publish the module locally:

```
Bash
spacetime build
spacetime publish debris_tracker
```

Step 2: Read & Write from Python using spacetimedb-sdk

Generate Python client bindings directly from your compiled C++ module:

```
Bash
spacetime generate --lang python --out-dir ./module_bindings
```

Your Python script parses Gunter's Space Page, runs your ML model, and directly reads/writes to SpacetimeDB:

```
Python
import asyncio
from spacetimedb_sdk.spacetimedb_async_client import SpacetimeDBAsyncClient
import module_bindings
from module_bindings.debris_voxel import DebrisVoxel
import module_bindings.trigger_breakup_reducer as trigger_breakup_reducer

client = SpacetimeDBAsyncClient(module_bindings)

# READ: Local cache update listener (triggered whenever C++ mutates a row)
def on_voxel_change(row_op, old_voxel, new_voxel, reducer_event):
    if row_op == "update":
        # Python accesses updated 3D ECI positions calculated by the C++ engine
        print(f"Voxel {new_voxel.voxel_id} shifted to ECI: ({new_voxel.x:.2f}, {new_voxel.y:.2f}, {new_voxel.z:.2f})")

# WRITE: Callback executed upon WebSocket handshake
def on_connect():
```

```

print("Python connected to SpacetimeDB!")

# Register table listener for real-time reads
DebrisVoxel.register_row_update(on_voxel_change)

# Execute the C++ reducer passing ML-derived satellite specs
# This writes new rows straight to the shared database
trigger_breakup_reducer.trigger_breakup(
    event_id=2024001,
    x=6800.0,
    y=1200.0,
    z=0.0,
    mass_kg=6600.0,
    EMR=45.0
)

async def main():
    await client.run(
        auth_token="",
        host_name="http://localhost:3000",
        module_address="debris_tracker",
        on_connect=on_connect,
        # Subscribe Python to the DebrisVoxel table cache
        queries=["SELECT * FROM DebrisVoxel"]
    )

if __name__ == "__main__":
    asyncio.run(main())

```

Step 3: Multi-User Interaction & Instant Sync

To make the app interactive across multiple clients (web users, simulation agents, or remote team members):

1. **State Synchronization:** Any query executed by a client (SELECT * FROM DebrisVoxel) populates an in-memory client-side cache.
- 2.
3. **Delta Updates:** When Python or a C++ reducer modifies a row, SpacetimeDB computes the minimal bitwise delta and broadcasts it to all connected subscribers over WebSockets.
- 4.
5. **Multiplayer Control:** Users or autonomous ML agents can call reducers (such as inject_satellite() or simulate_avoidance_maneuver()) simultaneously. Transactions process atomically on the server inside the C++ engine runtime.
- 6.

Execution Steps for the Team

1. **Host Server Module:** Run `spacetime dev` to launch the local SpacetimeDB instance and auto-recompile `lib.cpp` when code changes.
- 2.
3. **Train ML & Feed Data (Python):** Use Python to run your historical breakup model and invoke `trigger_breakup`.
- 4.
5. **Propagate State (C++ Module):** Let the scheduled C++ reducer update physical state at 60 Hz on the server.
- 6.
7. **Subscribe (Web/Clients):** Point your React/CesiumJS or Unreal Engine client at `ws://localhost:3000` with `SELECT * FROM DebrisVoxel` to stream live rendering updates without building custom API endpoints.

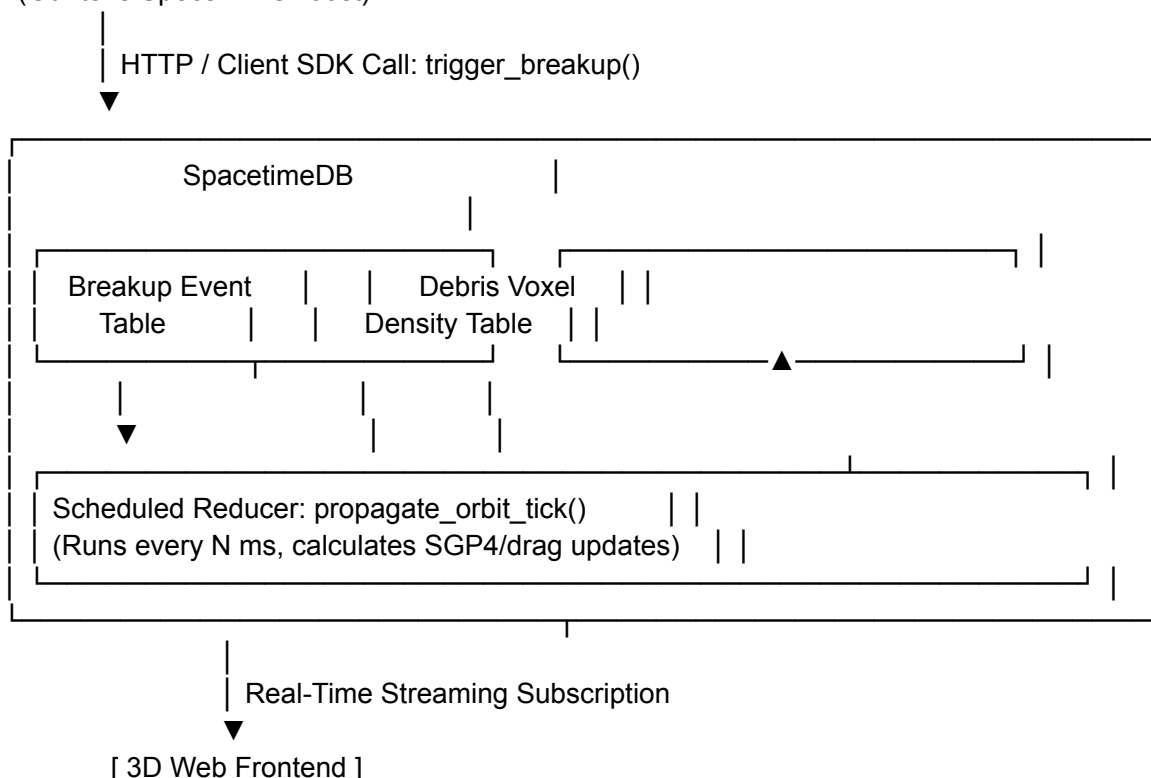
Using SpacetimeDB as your backend for this project is a massive hackathon advantage. Because SpacetimeDB combines **in-memory database storage**, **server application logic (reducers)**, and **real-time WebSocket subscriptions** into a single WASM module, you do not need to build a separate REST API, setup WebSockets, or configure an external database.

SpacetimeDB becomes both your physics execution engine and your streaming server. As your simulation updates orbital positions in the database, SpacetimeDB automatically streams those 3D changes straight to your frontend dashboard in real-time.

High-Level Architecture

[Python ML Pipeline]

(Gunter's Specs + XGBoost)



1. Define the SpacetimeDB Schema (Rust Module)

In your SpacetimeDB module (written in Rust or C#), define tables for breakup events and spatial debris voxels. Marking tables as `public` allows connected web clients to query and subscribe to them in real-time.

Rust

```
use spacetimedb::{table, reducer, ReducerContext, Schedule};
```

```
// Stores the metadata for each breakup event
```

```
#[table(name = breakup_event, public)]
```

```
pub struct BreakupEvent {
```

```
    #[primary_key]
```

```
    pub event_id: u64,
```

```
    pub sat_name: String,
```

```
    pub mass_kg: f32,
```

```
    pub emr_j_per_g: f32,
```

```
    pub pos_x: f32,
```

```
    pub pos_y: f32,
```

```
    pub pos_z: f32,
```

```
}
```

```
// Stores 3D spatial voxel density (particles / km³)
```

```
#[table(name = debris_voxel, public)]
```

```
pub struct DebrisVoxel {
```

```
    #[primary_key]
```

```
    pub voxel_id: u64,
```

```
    pub event_id: u64,
```

```
    pub x: f32, // Earth-Centered Inertial (ECI) km
```

```
    pub y: f32,
```

```
    pub z: f32,
```

```
    pub vx: f32, // Mean orbital velocity vector
```

```
    pub vy: f32,
```

```
    pub vz: f32,
```

```
    pub density: f32, // Particle concentration
```

```
}
```

```
// Internal schedule table to drive the simulation loop
```

```
#[table(name = simulation_timer)]
```

```
pub struct SimulationTimer {
```

```
    #[primary_key]
```

```
    pub scheduled_id: u64,
```

```
    pub scheduled_at: ScheduleAt,
```

```
}
```

2. Implement the Physics & Event Reducers

Reducers act as your server functions. When a breakup is triggered, the reducer initializes the voxel density cloud and starts the periodic orbital propagation loop.

```
Rust
#[reducer]
pub fn trigger_breakup(
    ctx: &ReducerContext,
    event_id: u64,
    sat_name: String,
    mass_kg: f32,
    emr: f32,
    initial_pos: (f32, f32, f32),
    initial_vel: (f32, f32, f32)
){
    // 1. Record the event
    BreakupEvent::insert(BreakupEvent {
        event_id,
        sat_name,
        mass_kg,
        emr_j_per_g: emr,
        pos_x: initial_pos.0,
        pos_y: initial_pos.1,
        pos_z: initial_pos.2,
    });

    // 2. Generate initial Debris Voxels using NASA EVOLVE / DebrisSat physics
    // For a hackathon demo, instantiate N representative voxel clusters
    for i in 0..100 {
        DebrisVoxel::insert(DebrisVoxel {
            voxel_id: (event_id * 1000) + i,
            event_id,
            x: initial_pos.0 + (i as f32 * 2.0),
            y: initial_pos.1 + (i as f32 * 1.5),
            z: initial_pos.2,
            vx: initial_vel.0 + (i as f32 * 0.01),
            vy: initial_vel.1,
            vz: initial_vel.2,
            density: 50.0 / (1.0 + i as f32),
        });
    }

    // 3. Schedule the simulation loop to run every 100ms
    ctx.schedule_reducer(
        step_simulation,
        std::time::Duration::from_millis(100)
    )
}
```

```

    );
}

// Scheduled Reducer: Runs automatically inside the DB to update positions
#[reducer]
pub fn step_simulation(ctx: &ReducerContext, _timer: SimulationTimer) {
    let dt = 0.1; // Simulation step (seconds or accelerated time)

    // Loop over all voxels and apply orbital mechanics (J2 + velocity step)
    for mut voxel in DebrisVoxel::iter() {
        // Update positions based on velocity vectors
        voxel.x += voxel.vx * dt;
        voxel.y += voxel.vy * dt;
        voxel.z += voxel.vz * dt;

        // Save updated position back to DB
        // SpacetimeDB automatically streams this update to all web clients
        DebrisVoxel::update_by_voxel_id(&voxel.voxel_id, voxel);
    }

    // Re-schedule next tick
    ctx.schedule_reducer(
        step_simulation,
        std::time::Duration::from_millis(100)
    );
}

```

3. Connect Python ML Pipeline to SpacetimeDB

Your Python script parses Gunter's Space Page and runs the XGBoost model to predict total mass, energy release, and fragment expansion factors. Once calculated, Python calls the SpacetimeDB reducer directly via the client SDK or HTTP API:

```

Python
import requests

# SpacetimeDB endpoint
SPACETIMEDB_URL = "http://localhost:3000/database/v1/debris_tracker/call/trigger_breakup"

# Results output from Python ML model
payload = {
    "args": [
        2024001,      # event_id
        "Intelsat 33e", # sat_name
        6600.0,       # dry_mass kg
        45.0,         # EMR J/g
        [6800.0, 1200.0, 0.0], # Initial ECI position (x, y, z)
    ]
}

```

```

    [0.0, 7.5, 0.2]    # Initial velocity vector
  ]
}

# Post directly to database reducer
response = requests.post(SPACETIMEDB_URL, json=payload)
print("Breakup initialized in SpacetimeDB:", response.status_code)

```

4. Direct Client Subscription in Frontend (React / CesiumJS)

In your web dashboard, you do **not** write `fetch()` calls or handle polling loops. You subscribe to the `DebrisVoxel` table using SpacetimeDB's TypeScript SDK.

When `step_simulation()` mutates rows in the database, SpacetimeDB pushes the new 3D coordinates straight to your UI.

```

TypeScript
import { useState, useEffect } from 'react';
import { DbConnection } from './module_bindings'; // Generated by SpacetimeDB CLI

export function DebrisTracker3D() {
  const [voxels, setVoxels] = useState([]);

  useEffect(() => {
    // 1. Connect to SpacetimeDB
    const conn = DbConnection.builder()
      .withUri('ws://localhost:3000')
      .withModuleName('debris_tracker')
      .build();

    // 2. Subscribe to voxel updates
    conn.subscriptionBuilder()
      .onApplied(() => console.log("Subscribed to cloud density stream!"))
      .subscribe(["SELECT * FROM DebrisVoxel"]);

    // 3. React instantly when rows are updated by the database loop
    conn.db.debrisVoxel.onUpdate((oldVoxel, newVoxel) => {
      setVoxels(prev =>
        prev.map(v => v.voxelId === newVoxel.voxelId ? newVoxel : v)
      );
    });

    conn.db.debrisVoxel.onInsert((newVoxel) => {
      setVoxels(prev => [...prev, newVoxel]);
    });
  }, []);
}

```

```

return (
  <CesiumGlobeOrDeckGL
    cloudData={voxels} // Renders 3D voxels/heatmaps continuously
  />
);
}

```

Key Hacks for Demo Day

1. **Time Compression:** Space moves fast, but simulation visualizations look better accelerated. In your `step_simulation` reducer, multiply `dt` by a speed factor (e.g., $1\text{secondrealtime} = 10\text{minutesorbitaltime}$) so judges can watch a localized explosion turn into a full orbital ring in under 30 seconds.
- 2.
3. **Limit Voxel Granularity:** SpacetimeDB can handle high throughput, but WebGL rendering in browsers can lag if you stream tens of thousands of individual points per frame. Keep your active voxel count between **200 and 1,000 spatial voxels** per cloud, with each voxel storing an aggregate density metric.
- 4.
5. **Use the SpacetimeDB CLI:** Run `spacetime dev` during development. It automatically re-compiles your Rust module on save and auto-generates matching TypeScript types for your frontend.

****Yes.**** You can read and write to SpacetimeDB using Python.

While you can't author *server-side module logic* in Python, you can use the official `spacetimedb-sdk` Python library to act as a client.

****Writing**** from Python is done by calling ****reducers**** (server-side functions) defined on your SpacetimeDB module.

****Reading**** in Python is done by subscribing to tables (``SELECT * FROM ...``), which SpacetimeDB syncs to an in-memory local cache in Python.

Step 1: Install & Generate Python Bindings

1. Install the SDK via PyPI:

```

``bash
pip install spacetimedb-sdk

```

...

2. Generate client bindings for Python using the SpacetimeDB CLI:

```
```bash
spacetime generate --lang python --out-dir ./module_bindings
```

```
```
```

```
---
```

Step 2: Python Code Example (Reading & Writing)

Here is how your Python ML script can connect, subscribe (read), and trigger reducers (write):

```
```python
import asyncio
from spacetimedb_sdk.spacetimedb_async_client import SpacetimeDBAsyncClient
import module_bindings # Auto-generated bindings folder
from module_bindings.debris_voxel import DebrisVoxel
import module_bindings.trigger_breakup_reducer as trigger_breakup_reducer

Initialize client
spacetime_client = SpacetimeDBAsyncClient(module_bindings)

1. READ: Event listener when local cache updates from the DB
def on_voxel_update(row_op, old_voxel, new_voxel, reducer_event):
 if row_op == "insert":
 print(f"New Voxel added at ECI ({new_voxel.x}, {new_voxel.y}, {new_voxel.z}) with density {new_voxel.density}")

2. Connection callback
def on_connect():
 print("Connected to SpacetimeDB!")

Register the row update callback for reads
DebrisVoxel.register_row_update(on_voxel_update)

WRITE: Call a reducer to insert/mutate data in SpacetimeDB
(e.g., passing output from your XGBoost / Physics model)
trigger_breakup_reducer.trigger_breakup(
 event_id=2024001,
 sat_name="Intelsat 33e",
 mass_kg=6600.0,
 emr=45.0,
```

```

 pos_x=6800.0,
 pos_y=1200.0,
 pos_z=0.0
)

```

# 3. Start the async client loop

```

async def main():
 await spacetime_client.run(
 auth_token="", # Optional token if auth is enabled
 host_name="http://localhost:3000",
 module_address="debris_tracker",
 on_connect=on_connect,
 queries=["SELECT * FROM DebrisVoxel"] # Subscribe to table for reads
)

```

# READ: You can also iterate over the local synchronized cache at any time

```

for voxel in DebrisVoxel.iter():
 print(f"Voxel ID: {voxel.voxel_id}, Density: {voxel.density}")

```

```

if __name__ == "__main__":
 asyncio.run(main())

```

...

---

### ### Key Takeaway for Your Architecture

**\*\*Python as the Driver:\*\*** Your Python script acts as the heavy-lifting ML pipeline that scrapes metadata, predicts fragment counts, calculates  $A/M$  spreads, and fires calls to SpacetimeDB via ``trigger_breakup()``.

**\*\*SpacetimeDB as the Real-Time Sync:\*\*** SpacetimeDB receives those reducer calls, executes the state updates, and broadcasts the updated 3D coordinates out to your React / WebGL frontend automatically.